

**MTE544 – Autonomous Mobile Robotics**

University of Waterloo

Department of Mechanical and Mechatronics Engineering

# Lab 2: Closed loop control of mobile robots



| **Group member (Family Name, First Name)** | **Student ID** |
|---|---|
| Adams, Hunter | 20997503 |
| Chen, Joe | 20998545 |
| Wielowieyski, Jacob | 21004619 |



**Station number:** 151

**Robot number:** 9 and 16

November 4, 2026

## 1 Logged errors, tuning process and trajectories

**Setup.** TurtleBot 4, internal odometry as the position sensor, 10 Hz control loop. The distance error $e_d$ is the distance to the goal and the heading error $e_\theta$ is the bearing to the goal minus $\theta$, wrapped to $[-\pi, \pi]$; $\dot{e}$ is the slope over the last three samples and $\int e$ uses the trapezoid rule on the odometry time stamps. Commands are limited to 0.22 m/s and 1.9 rad/s. For the point controller we deliberately kept the gains low enough that the commands stay below these limits, so that the linear behaviour of each controller can be observed; a saturated controller only shows the limit. Goal tolerance: 1 cm for point runs, 5 cm for trajectories. Each run is named by its start time (e.g. 17:12:26). Low batteries cost us lab time: we changed robots several times and waited for charging. Due to limited time, the TAs told us to skip PID point runs, so the point-controller results on the robot are for P. Our PID is one controller throughout: $k_{lp} = 0.2$, $k_{lv} = 0.05$, $k_{li} = 0.025$, $k_{ap} = 1.0$, $k_{av} = 0.2$, $k_{ai} = 0.025$, run on the robot for the parabola (18:17:44) and the sigmoid (18:27:45), and simulated for the point goal (Fig. 1, Table 1).

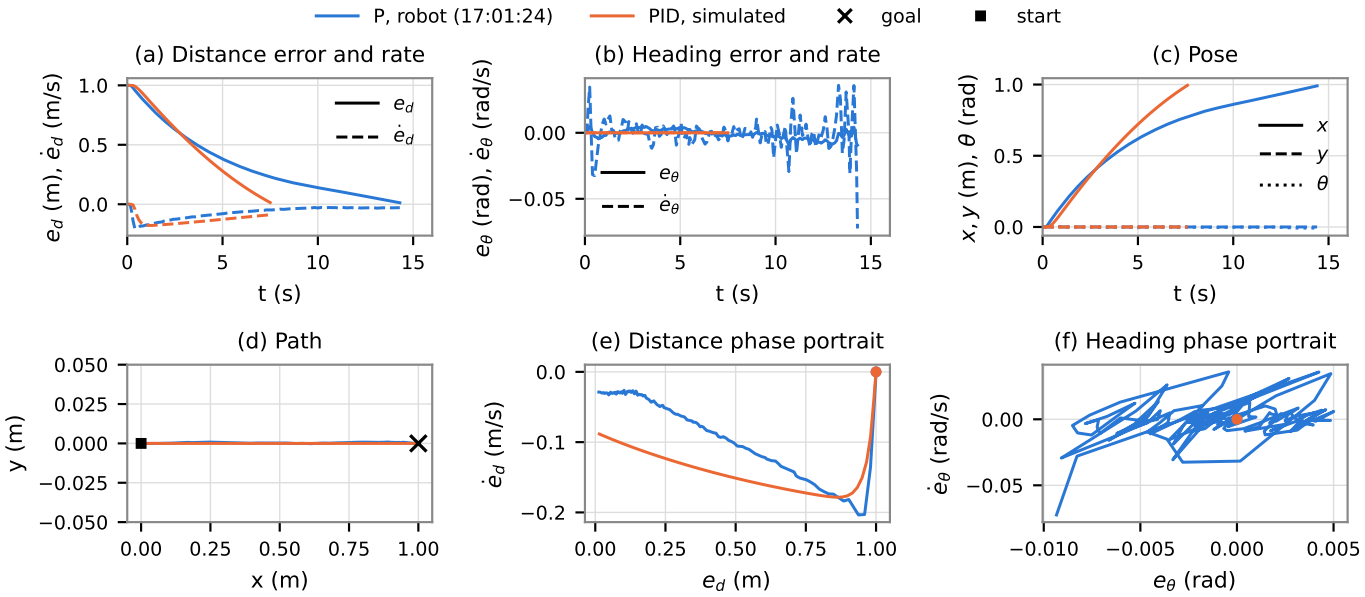



**Figure 1:** Point controller, goal 1 m straight ahead, $k_{lp} = 0.2$ (heading error below 0.6°, so the heading gains play no part). Blue: P on the robot (17:01:24). Orange: our PID, same controller code, **simulated, not measured**, on a robot model fitted to run 17:12:26 (`sim_point.py`). (a, b) $e$ and $\dot{e}$ vs time, (c) $x$, $y$, $\theta$ vs time, (d) path, (e, f) $\dot{e}$ vs $e$. Neither command reaches the 0.22 m/s limit (PID peaks at 0.207 m/s). P slows in proportion to $e_d$ and stops 9 mm short of the goal at 0.03 m/s after 14.4 s; the integral keeps PID faster: within 1 cm after 7.6 s, still at 0.09 m/s. After the stop command the model rolls on for its delay and lag, after the logs end: PID ends ≈3 cm past the goal, P ≈0.5 cm. With a goal behind the robot the distance integral winds up during the turn and the simulated PID circles the goal, so we avoided it.

**Tuning process (Fig. 2).** *(a) Distance gain.* When driving 1 m straight ahead with unsaturated gains, $e_d$ decays exponentially as predicted by a theoretical first-order model: fitted time constants 9.8 s at $k_{lp} = 0.1$ and 5.0 s at 0.2 ($1/k_{lp} = 10$ and 5 s). The last part of the graph turns into a straight line because the robot does not drive slower than about 0.03 m/s (lower end saturation is hit). To find where upper saturation starts we raised $k_{lp}$: from 0.3 the command starts at the limit and quickly drops, and from 2.5 it stays there until the last 9 cm (4.8 s to the goal whatever the gain). We chose $k_{lp} = 0.2$, the largest gain that keeps $v = k_{lp} e_d$ below 0.22 m/s for our 1 m goals. *(b) Heading gain, goal behind the robot.* For a 180° turn, $\omega = k_{ap} e_\theta$ stays below 1.9 rad/s only for $k_{ap} < 0.6$. With 0.4 and 0.5 the heading decays without overshoot (fitted time constants 4.1 and 2.6 s vs $1/k_{ap} = 2.5$ and 2 s; slower than the ideal loop because the bearing to the goal keeps moving while the robot drives sideways). This is because a lower $k_{ap}$ drives a wider arc (627 and 491 mm), since the distance loop drives forward during the turn. A saturated run ($k_{ap} = 2.5$) turns faster (1.15 s rise, 196 mm arc) but overshoots and hides the linear dynamics. We chose $k_{ap} = 0.5$ for point

goals behind the robot (run 17:12:26).

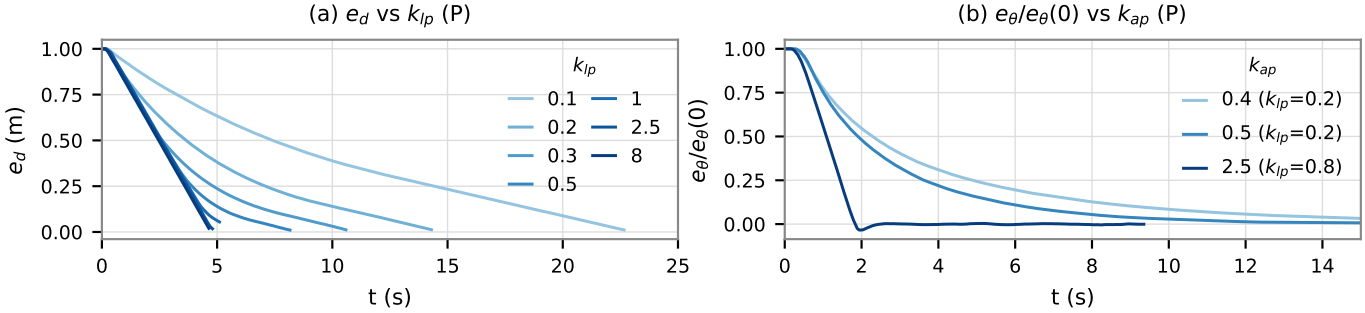



**Figure 2:** Tuning on the robot. (a) $e_d$ for seven values of $k_{lp}$ (P, goal 1 m ahead). (b) Normalised heading error for three values of $k_{ap}$ (P, goal behind; the 2.5 run used $k_{lp} = 0.8$, the others 0.2).

**Trajectories and choice of controller (Fig. 3).** Most of our trajectory tuning went into the parabola (16 runs). Because $e_d$ is measured to the end of the curve, P slows down over the last $v_{\max}/k_{lp} = 1.1$ m. With the same $k_{lp}$ and $k_{ap}$, P, PI and our PID tracked the parabola similarly well (RMS 13–15 mm, worst error in the tight first 0.5 m), but the integral term cut the time to come within 5 cm of the end from 21.3 s to 13.3–13.6 s: it holds the speed command at the 0.22 m/s limit for the whole run, so the robot does not slow down near the end. The D term gave the lowest worst-case deviation of the three (34 mm vs 37 mm for PI and 40 mm for P), so we chose PID for both trajectories. With a 1 cm tolerance a PI run (17:53:25) circled the end of the parabola: the integral keeps $v = 0.22$ m/s, and the tightest turn at that speed, $v_{\max}/\omega_{\max} = 0.12$ m, cannot hit a 1 cm circle, so the trajectory tolerance was set to 5 cm. We stretched the sigmoid to $y = 2/(1+e^{-x}) - 1$, $x \in [0, 4]$ (lab: $2/(1+e^{-2x}) - 1$, $x \in [0, 2.5]$), so that it leaves the origin at 27° instead of 45°: the robot starts facing $+x$ and drives forward while it turns. Its worst error (59 mm) is still at the start ($x = 0.3$ m); the RMS error is 20 mm.

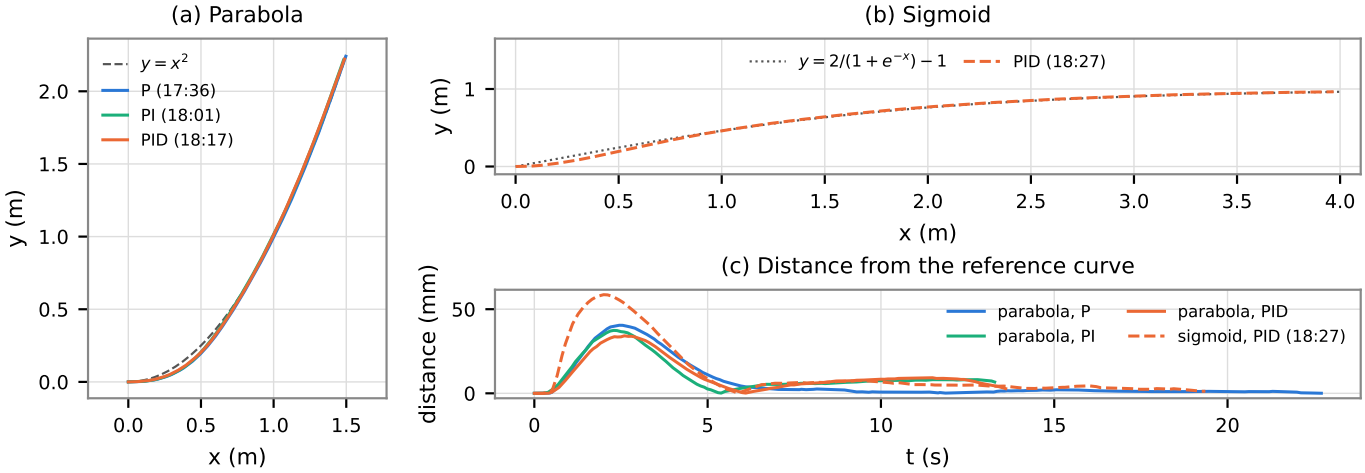



**Figure 3:** Trajectories on the robot. (a) Parabola $y = x^2$, $x \in [0, 1.5]$, with P (17:36:05), PI (18:01:56) and our PID (18:17:44) at the same $k_{lp} = 0.2$, $k_{ap} = 1.0$. (b) Sigmoid, stretched to $y = 2/(1+e^{-x}) - 1$, $x \in [0, 4]$, with our PID (18:27:45). (c) Distance from the robot to the reference curve. The P run used a 1 cm end tolerance (PI, PID: 5 cm); it was within 5 cm of the end after 21.3 s.

## 2 P vs PID point controller: stability analysis and discussion

**Stability analysis.** *Distance loop.* With the heading aligned, $\dot{e_d} = -v$, so the robot itself is an integrator. P control gives $\dot{e_d} = -k_{lp} e_d$: first order, time constant $1/k_{lp}$, no overshoot and zero steady-state error to a fixed goal. Our runs closely match: fitted time constants 9.8, 5.0 and 3.2 s for $k_{lp} = 0.1$, 0.2 and 0.3 (theoretical 10, 5 and 3.3 s; the 0.3 run is at the speed limit only for its first 1.5 s). *Heading loop.* The robot starts moving 0.2–0.6 s after the first command (median 0.3 s); modelling this as a lag $\tau \approx 0.2$ s gives

**Table 1:** Point controller, goal 1 m straight ahead, $k_{lp} = 0.2$. P: measured on the robot (run 17:01:24). **PID: our PID, simulated, not measured** (PID point runs were skipped on the TAs' recommendation due to limited lab time): the same controller code on a robot model fitted to run 17:12:26. Rise time: distance from 90 % to 10 % of its start value. Settling: last exit from ±5 %. Times from the first command. Percent overshoot: peak past the goal of the linear model of the distance loop (`expected_pid.py`, below), which keeps controlling past the goal. The logs end inside the 1 cm tolerance; the final position adds the model's roll after the stop command (delay + lag). The heading error stays below 0.6°, so only the distance loop is compared.

| | Metric | P, measured | PID, simulated |
|---|---|---|---|
| Agility | Rise time (s) | 10.6 | 5.6 |
| | Settling time, 5 % (s) | 13.1 | 7.2 |
| | Time to goal (s) | 14.4 | 7.6 |
| Accuracy | Distance to goal at the stop command (mm) | 9.3 | 5.6 |
| | Steady-state error | 0 | 0 |
| Overshoot | Percent overshoot, linear model (%) | 0 | 25 |
| | Final position past the goal, est. (% of 1 m) | 0.5 | 3.3 |
| | Speed at the goal (m/s) | 0.03 | 0.09 |

$\tau\ddot{e}_\theta+(1+k_{av})\dot{e}_\theta+k_{ap}e_\theta = 0$ and $\zeta = (1+k_{av})/(2\sqrt{k_{ap}\tau})$. Our P gains $k_{ap} = 0.4$–$0.5$ give $\zeta = 1.8$–$1.6$: overdamped, and we measured no overshoot. As a check on $\tau$, the saturated $k_{ap} = 2.5$ run ($\zeta = 0.71$, predicted 4.3 %) overshot by 3.4 %. *Our PID.* With the I term the distance loop becomes $(1 + k_{lv})s^2 + k_{lp}s + k_{li} = 0$: $\zeta = 0.62$ for our $k_{li} = 0.025$, so the overdamped P loop becomes slightly oscillatory ($\zeta \geq 0.7$ would need $k_{li} \leq 0.019$). In this linear model $e_d$ reaches zero after 7.3 s at 0.08 m/s and then overshoots by 25 %; the simulation in Table 1 agrees on the arrival (7.6 s, 0.09 m/s), and stopping at the 1 cm tolerance cuts the overshoot to the roll after the stop command. In the heading loop ($k_{ap} = 1.0$) P alone gives $\zeta = 1.12$; the D term raises it to 1.34, and the I term adds a slow mode (time constant 39 s) and a 2.6 % overshoot on a 180° turn. The two loops are coupled, though: with a goal behind the robot the distance integral winds up during the turn, and the simulated PID circles the goal at full speed instead of stopping.

**Discussion.** *Agility:* the I term makes the distance loop about twice as fast (rise 5.6 vs 10.6 s, goal after 7.6 vs 14.4 s): as $e_d$ shrinks, the integral keeps the speed up where P slows down in proportion to $e_d$. On the parabola the same effect cut the time from 21.3 to 13.6 s. *Accuracy:* the robot integrates the speed command, so P already has zero steady-state error to a fixed goal; both controllers stop inside the 1 cm tolerance, and the I term adds no accuracy. *Overshoot:* PID arrives at 0.09 m/s instead of 0.03 m/s, so it ends ≈3 cm past the goal instead of ≈0.5 cm, and the linear model predicts a 25 % overshoot if it did not stop. It also fails for a goal behind the robot.

For a fixed goal we would therefore use P, or PID only with an integral clamp and the goal ahead of the robot; we would add D only if a higher $k_{ap}$ made the heading underdamped (above $k_{ap} \approx 1.25$ for $\tau = 0.2$ s). For trajectories PID is the better choice: the speed the integral adds shortens the run without hurting tracking, and the 5 cm end tolerance absorbs the arrival speed. The wide arc at the start of the sigmoid comes from driving while turning; scaling the speed by $\cos e_\theta$ would remove it. P with $k_{ap} = 3.0$ tracked the parabola tightest (8 mm, 23.1 s), so a higher $k_{ap}$ is the next PID tuning step.